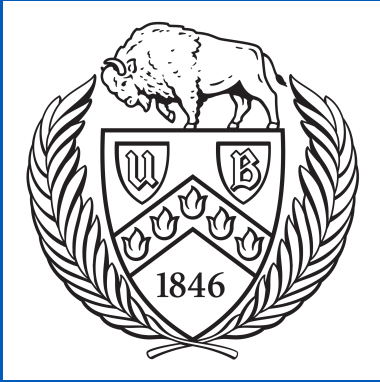




Final Report

CSE 526 - Blockchain Application Development
Team Name - Beacon

Jayesh Kishor Suryavanshi
UB Person Number - 50417114 // jayeshki@buffalo.edu
Fardin Apurbo
UB Person Number - 50237731 // fakhan4@buffalo.edu

1 ABSTRACT

A peer-to-peer network of shared EV chargers, the Beacon app will open up private charging stations like homes, restaurants, garages for public use, immediately increasing the number of reliable charging places without requiring expensive new installations. Anyone can list their home charger for rent on Beacon to earn extra money (ERC20 tokens) and help others drive electric without any worries. People who rent out their chargers will be called hosts. The hosts in a particular area can set availability and prices (as ERC20 tokens) on the web application and the EV owner who wants a quick, reliable charge can see the hosts' availability and charging options on the web application. The EV owner will pay based on the amount of charging done to the host's decentralised address. This is a pay-as-you-go model that enables EV owners to make a one-time purchase of a single charge which is paid using an ERC20 token.

2 ISSUES ADDRESSED

The advent of electric vehicles is the most significant initiative towards going green and making our environment cleaner. With the constant advancements in the automotive industry, the Electric vehicle market has experienced a massive development over the past couple of years. And the growing demand for a greener environment has raised the need for creating EV charging infrastructure. However, an insufficient charging infrastructure affects a rapid adoption rate of EVs. At the same time, EV companies have refrained from developing an EV-charging infrastructure on a large scale because of the limited demand for EVs. People are continuing to use gas powered vehicles over EVs due to issues such as limited range, limited battery life, long charging times, and the lack of EV charging stations. Existing solutions to these problems, such as building more charging stations, increasing battery capacity, and road-charging have not been proven effective so far. Against this backdrop, we are proposing a peer-to-peer solution of charger sharing among EV owners, so that people would not have to worry about long detours in their trip just for charging their vehicles.

3 SOURCE OF INSPIRATION

On Sept 9, famous tech youtuber named 'MKBHD' decided to put a gas powered vehicle against a Tesla and a Ford Mustang Mach-E in a 1000 mile road trip from NYC to Buffalo-Niagara region (Niagara Falls) and back. The result was such that the gas powered vehicle had already reached the destination, The Tesla Model S was fortunate enough to get a full charge done because it had a supercharger in it's way. However, The Mustang Mach-E faced a lot of problems because it encountered broken and out-of-order charging stations in the way which delayed the travel by almost 4-5 hours after the two cars had already made it to the destination. This is the case for most of the electric vehicles at this moment in the country. Had there been a peer to peer charging network, where the electric car would have found a charging host in the way, it would have reached the destination much earlier.

Video Link: [Youtube/MKBHD](https://www.youtube.com/watch?v=MKBHD)

4 UML Use Case Diagram

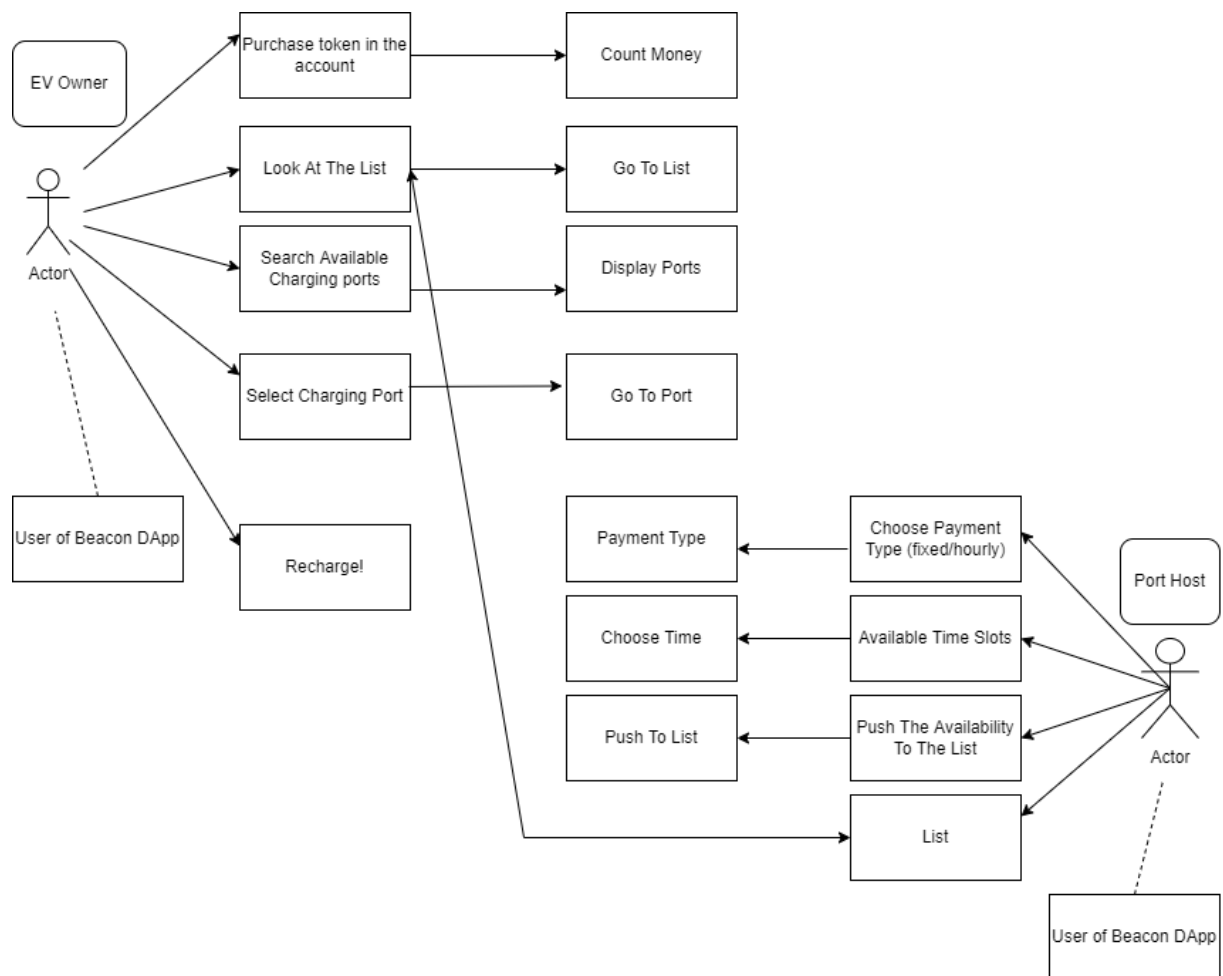


Figure 1: Beacon - UML Use Case Diagram

5 Wireframes

The wireframes are as follows:

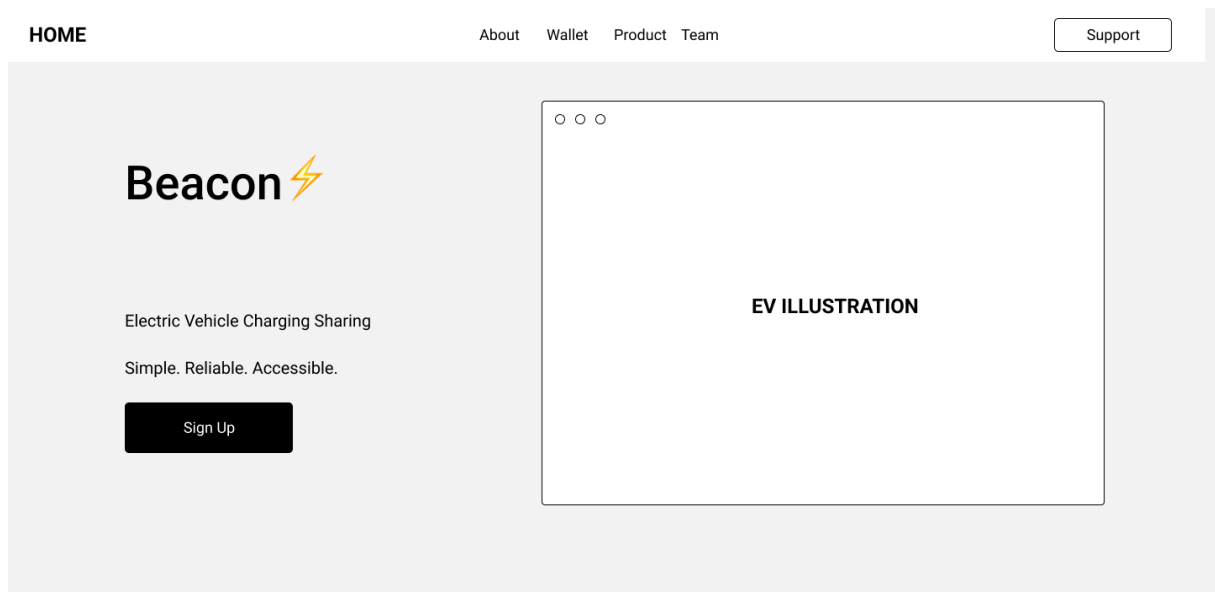


Figure 2: Wireframe 1

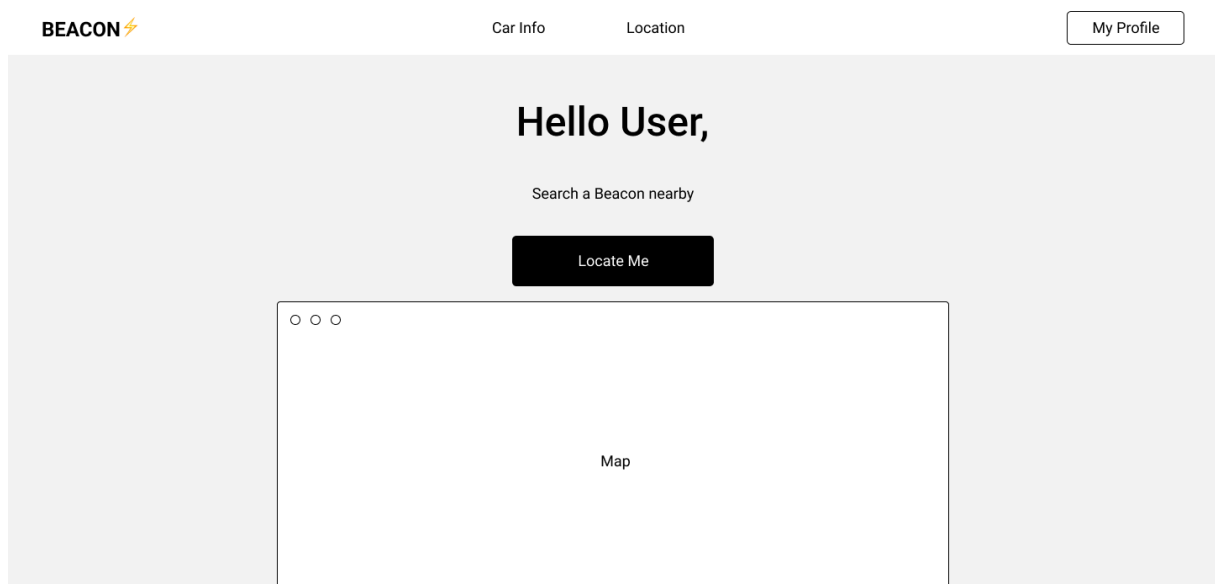


Figure 3: Wireframe 2

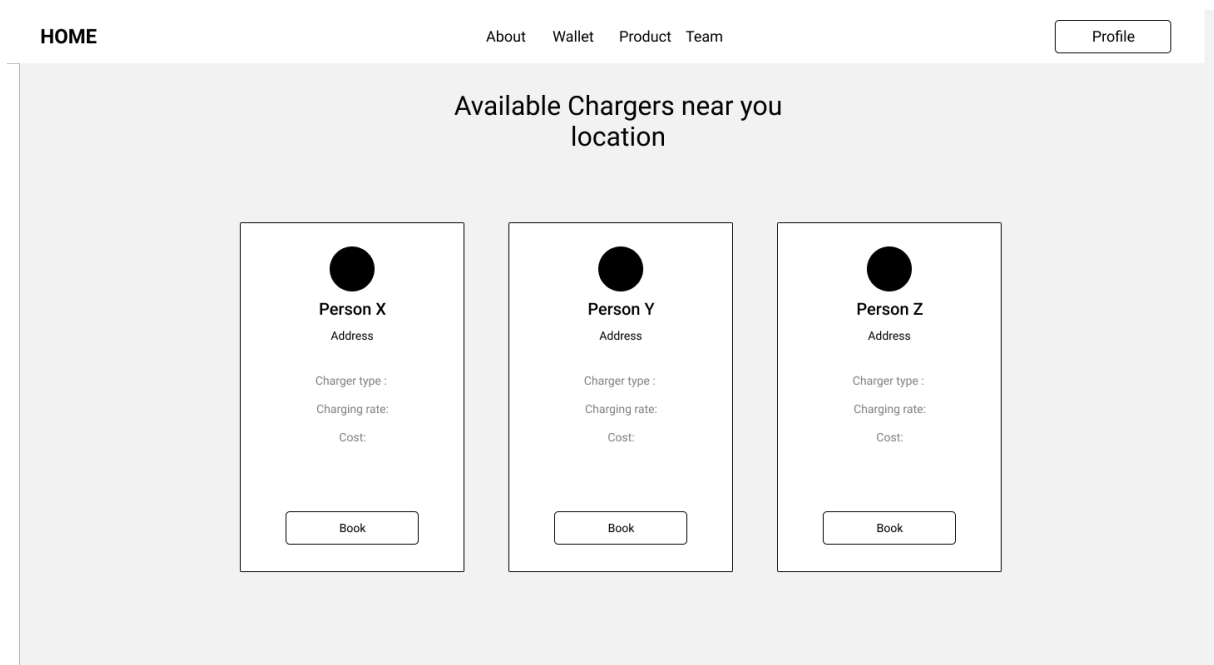


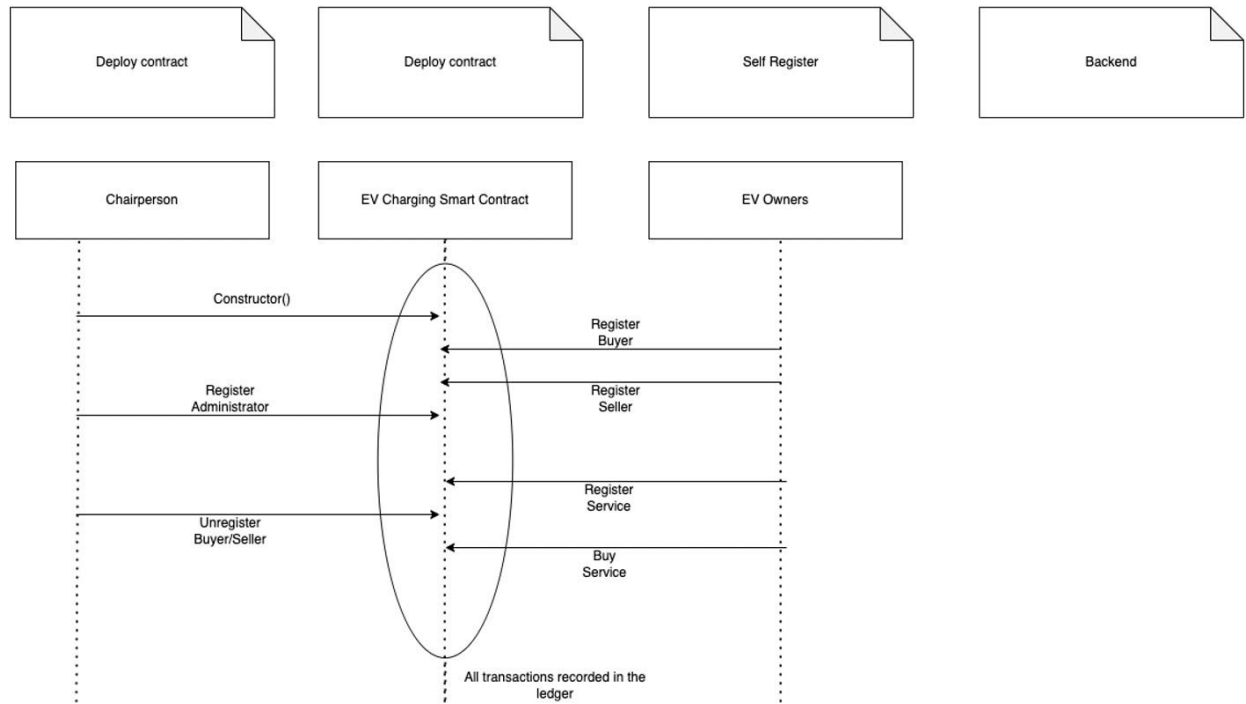
Figure 4: Wireframe 3

6. Diagrams

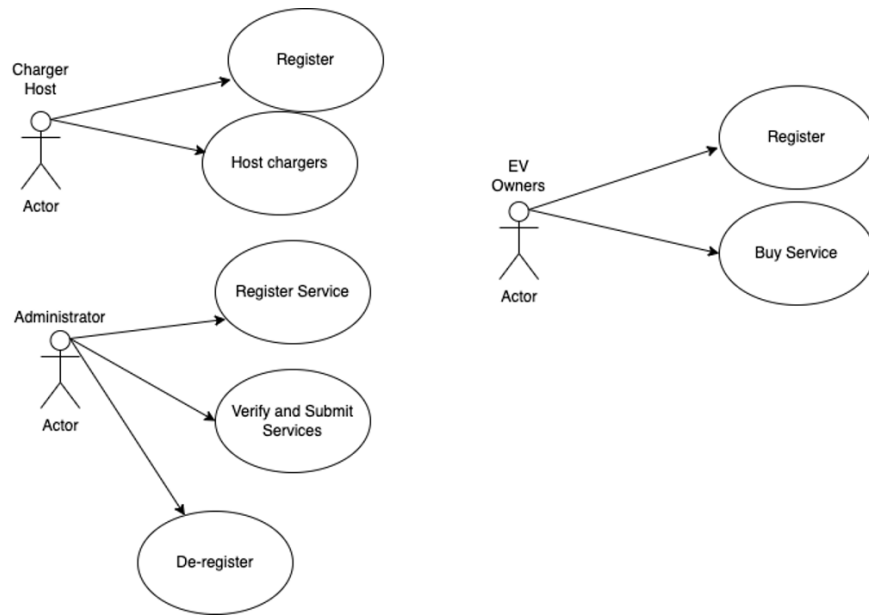
The Quad chart diagram is as follows:

Use Case: EV Marketplace Problem Statement: A Decentralized Marketplace for EV owners to have alternative choices to charge their vehicles.	Issues with existing centralized model: • There aren't many options to charge their vehicles. • Centralized systems have a lot of overhead charges • It can be organized more efficiently with the help of smart contracts • Smart contracts can have all the transactions recorded for as needed purposes in the future • Smart contracts take care of all the rules and regulations
Proposed Blockchain-based solution: • EV owners can request to provide their home chargers to be used for rent • EV owners will be able to charge their vehicles no matter where they are • Transactions are recorded on the ledger • Providing better services for the EV market	Benefits: • It is more efficient to own EVs • Charging would be as common as gas stations • It would be cheaper for consumers to use another source for charging their vehicles • It would provide an extra source of income for EV owners who have a charger at their house or business location • It is better for the environment as more and more people would be inclined to get EVs after seeing how much EV owners save on costs that regular cars aren't able to provide.

The Sequence diagram is as follows:



The UML diagram is as follows:



The Quad Chart diagram is as follows:

Use Case: EV Marketplace Problem Statement: A Decentralized Marketplace for EV owners to have alternative choices to charge their vehicles.	Issues with existing centralized model: • There aren't many options to charge their vehicles. • Centralized systems have a lot of overhead charges • It can be organized more efficiently with the help of smart contracts • Smart contracts can have all the transactions recorded for as needed purposes in the future • Smart contracts take care of all the rules and regulations
Proposed Blockchain-based solution: • EV owners can request to provide their home chargers to be used for rent • EV owners will be able to charge their vehicles no matter where they are • Transactions are recorded on the ledger • Providing better services for the EV market	Benefits: • It is more efficient to own EVs • Charging would be as common as gas stations • It would be cheaper for consumers to use another source for charging their vehicles • It would provide an extra source of income for EV owners who have a charger at their house or business location • It is better for the environment as more and more people would be inclined to get EVs after seeing how much EV owners save on costs that regular cars aren't able to provide.

The Contract Diagram is as follows:

EV Charging
Address administrator; Address contractAddress; Mapping(uint => charging) public chargingData; Mapping (address => user) public userData; userData[administrator] = newUser;
Modifier balanceCheck (address fromAdress) {...} Modifier onlyAdmin () {...} Modifier registeredUser (address userId) {...} Modifier verifyPrice(uint price) {...}
Constructor () pulic payable { } function register() public { } function deregister(address userId) public onlyAdmin registeredUser(userId) { } function viewBalance() public view returns(uint) { } function viewContractAddress() public view returns(address) { } function addBalance() external payable balanceCheck(msg.sender) { } function sendBalance(address payable toAddress, uint price) external { } function upload(uint chargingId, uint price) public returns(uint) { } function buyCharging(uint chargingId, address payable toAddress) external payable { } function like(uint chargingId) public registeredUser(msg.sender) { } function unlike(uint chargingId) public registeredUser(msg.sender) { }

Short Description of a few data structures/functions/modifiers:

- Administrator has the ability to unregister an account
- Mapping data to provide for charging data for the host and the seller
- The constructor() makes the moderator to whoever deploys the contract
- Modifier balanceCheck requires enough credits for the purchase or else it reverts
- Modifier onlyAdmin() requires functions to be accessed only by the Administrator
- Modifier registeredUser() requires the user to be registered before accessing the application
- Modifier verifyPrice() requires the EV owner to send enough money for the charge, if it is lower, the transaction will revert
- Function register() is used to register people for either buying or selling the services
- Function deregister() is used only by the administrator to unregister an account
- Function viewBalance() is used to check the balance in the account, it is a view function therefore is not recorded in the blockchain
- Function addBalance is a payable function where you are able to put money in
- Function sendBalance is to transfer the amount and it requires that the transfer amount is at least equal to the amount needed.

Solidity Code after token implementation and deployment on Ropsten Test Network:

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity ^0.4.22 @8.9.0;
import "./../EV8Token.sol";

contract Charging{

    struct Charging{
        address owner;
        uint price;
        bool failed;
        int countLike;
    }

    struct user{
        bool isActive;
    }

    address administrator;
    address contractAddress;
    mapping(uint>Charging) public ChargingData;
    mapping(address>user) public userData;

    constructor(){
        administrator = msg.sender;
        contractAddress=address(this);
        user memory newUser = user(true);
        userData[administrator]=newUser;

        //initiate Charging values by admin
        Charging memory newCharging1 = Charging(msg.sender,10,true,0);
        ChargingData[1] = newCharging1;
        Charging memory newCharging2 = Charging(msg.sender,20,true,0);
        ChargingData[2] = newCharging2;
        Charging memory newCharging3 = Charging(msg.sender,30,true,0);
        ChargingData[3] = newCharging3;
        Charging memory newCharging4 = Charging(msg.sender,40,true,0);
        ChargingData[4] = newCharging4;
        Charging memory newCharging5 = Charging(msg.sender,50,true,0);
        ChargingData[5] = newCharging5;
        Charging memory newCharging6 = Charging(msg.sender,60,true,0);
        ChargingData[6] = newCharging6;
        Charging memory newCharging7 = Charging(msg.sender,70,true,0);
        ChargingData[7] = newCharging7;
        Charging memory newCharging8 = Charging(msg.sender,80,true,0);
        ChargingData[8] = newCharging8;
        Charging memory newCharging9 = Charging(msg.sender,90,true,0);
        ChargingData[9] = newCharging9;
    }

    // Modifier #####
    modifier balanceCheck(address fromAddress) {
        require(msg.value <= fromAddress.balance, "Insufficient balance");
        _;
    }

    modifier onlyAdmin(){
        require(msg.sender==administrator);
        _;
    }

    modifier registeredUser(address userID){
        require(userData[userID].isActive==true,"User is not registered for exchange");
        _;
    }

    modifier verifyPrice(uint price){
        require(msg.value==price,"Transfer value does not match the Charging price");
        _;
    }

    // Functions #####

    function register() public{
        user memory newUser = user(true);
        userData[msg.sender]=newUser;
    }

    function deregister(address userID) public onlyAdmin registeredUser(userID){
        user memory oldUser = userData[userID];
        userData[userID].isActive=false;
    }

    function viewBalance() public view returns(uint) {
        return address(this).balance;
    }

    function viewContractAddress() public view returns(address){
        return contractAddress;
    }

    function addBalance() external payable balanceCheck(msg.sender) {
    }

    function sendBalance(address payable toAddress, uint price) external {
        require(contractAddress.balance>price,"Smart Contract has Insufficient Balance");
        toAddress.transfer(price);
    }

    function upload(uint ChargingId, uint price) public returns(uint){
        Charging memory newCharging = Charging(msg.sender,price,true,0);
        ChargingData[ChargingId] = newCharging;
        return 1;
    }

    function buyCharging(uint ChargingId, address toAddress) public {
        // address fromAddress, address payable toAddress, uint price) external payable verifyPrice(price) balanceCheck(msg.sender)
        registeredUser(fromAddress) registeredUser(toAddress);
        //toAddress.transfer(ChargingData[ChargingId].price *10 ** 18);
        ChargingData[ChargingId].owner=msg.sender;
        ChargingData[ChargingId].isActive=false;
        WToken token = WToken(0x0000000000000000000000000000000000000000);
        token.transferFrom(msg.sender, toAddress, ChargingData[ChargingId].price *10 ** 18);
    }

    function like(uint ChargingId) public registeredUser(msg.sender){
        ChargingData[ChargingId].countLikes ++ 1;
    }

    function unlike(uint ChargingId) public registeredUser(msg.sender){
        ChargingData[ChargingId].countLikes -- 1;
    }

}
```

ERC-20 Contract Development/Deployment

Detailed explanation of ERC-20 functionality and methods

- An ERC-20 token is a fungible token where a fungible token is identical to every other token in value for the same class of tokens.
- An ERC-20 is like a currency, letting user buy certain utility or services by spending the tokens.
- The ERC-20 token definition includes the token name, symbol and an attribute that specifies how to represent a fractional of a token (like a decimal)
- These tokens piggyback on the Ethereum network and can operate with the same address as your Ethereum node.
- ERC-20 TOKEN can be exchanged for any other ERC-20 token on the crypto exchange.

Detailed explanation of how the ERC-20 contract connects to other contracts and uses the functionality

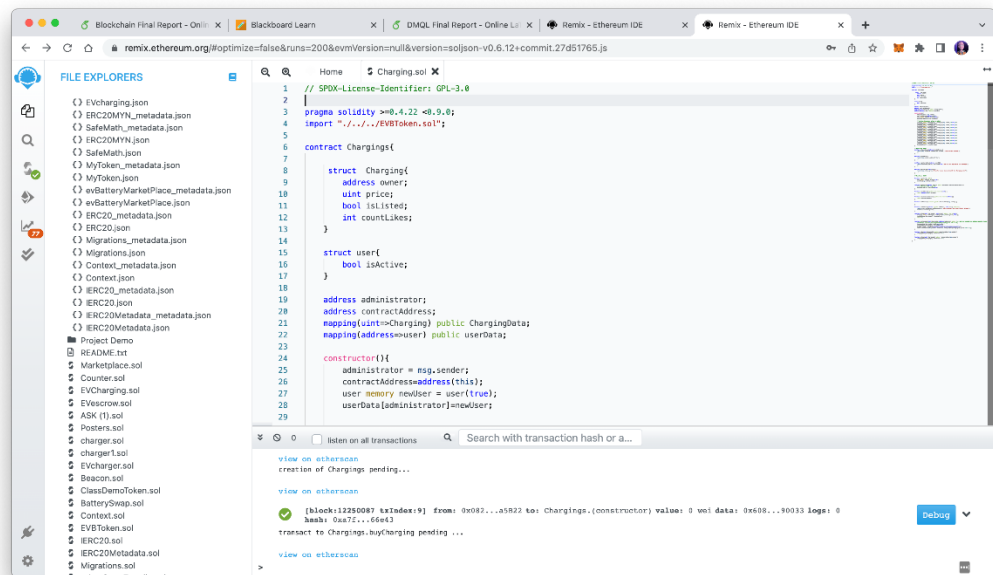
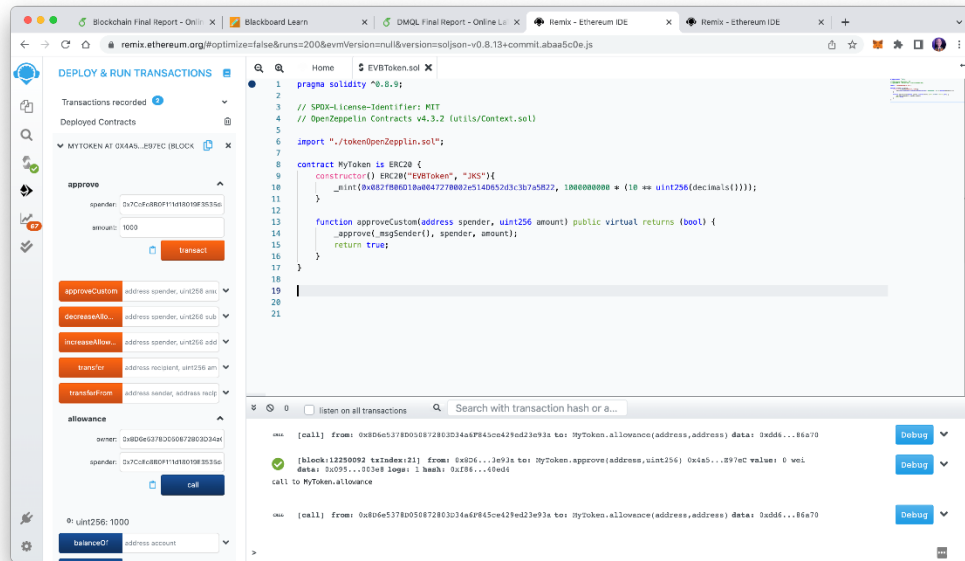
- We use solidity to create a contract for the ERC-20 using the Ethereum blockchain, which uses several functions and methods to create and use ERC-20 tokens to use in our marketplace.
- It effectively lets us use our token similarly to other more common cryptocurrency like Bitcoin or Ethereum itself
- The ERC-20 provides several functions that allow the smart contract to emulate a digital token.
- Some of ERC-20 defined functions are:
 - **totalSupply()** – it allows an instance of the contract to calculate and return the total amount of tokens that exists in circulation
 - **balanceof()** – It allows the smart contract to store and return the balances of the given addresses.
 - **approve()** – it gives the owner permission to allow the given address to withdraw instances of the token from the owners address.
 - **transfer()** – allows the owner to send token to another address just like a regular cryptocurrency like Bitcoin or Ethereum would.
 - **transferFrom()** – this function allows a smart contract to automate the transfer process and send a given amount of token on behalf of the owner.

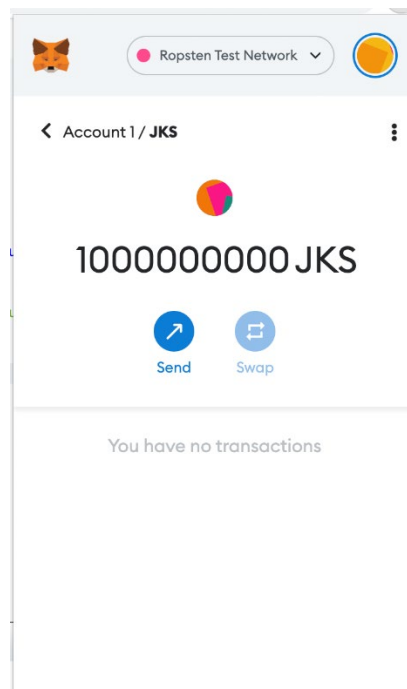
- We could also have a name for our token to let wallets identify the unique ERC-20 token
- We could also add a token symbol, essentially an abbreviation of the token name to help identify the token easily like BTC of Bitcoin or ETH of Ethereum.

Detailed explanation of the overall deployment process of your contracts and Dapp connection

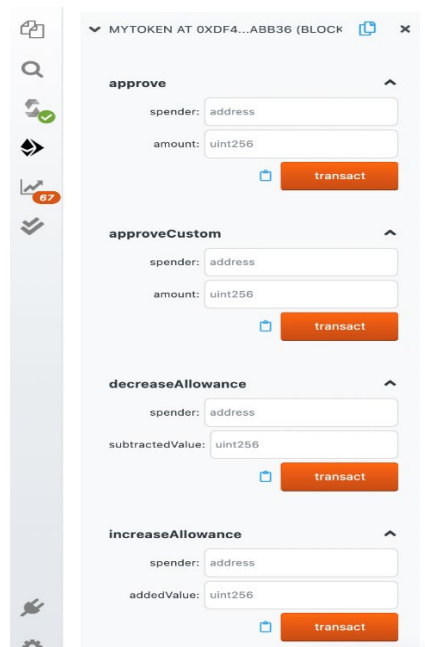
- We have our smart contract for the marketplace, and a smart contract for our ERC-20 token. The way we will connect the two contracts is with upgradable contracts. Since a smart contract is of immutable nature, we will use the ERC-20 contract inside of the marketplace contract by using a mechanism for delegating calls where a proxy can be deployed which points to the other contract. In our case, our contract for the marketplace will use this mechanism to use the ERC-20 contract for the use of ERC-20 tokens. Using this will help save time and space in blockchain as there will be no reusability of the same functions which will prevent the need to use of similar functions on the chain.

1. Deployed on Ropsten





2. Demonstrate interactions of methods



The screenshot shows a web interface for interacting with an ERC-20 token. On the left is a sidebar with icons for copy, search, wallet, transaction, graph, and settings. The main area contains four sections, each with a title and a set of input fields:

- transfer**: Fields for 'recipient: address' and 'amount: uint256'. An orange 'transact' button is at the bottom right.
- transferFrom**: Fields for 'sender: address', 'recipient: address', and 'amount: uint256'. An orange 'transact' button is at the bottom right.
- allowance**: Fields for 'owner: address' and 'spender: address'. A blue 'call' button is at the bottom right.
- balanceOf**: Field for 'account: address'. A blue 'call' button is at the bottom right.

A 'decimals' button is located at the bottom left of the main area.

3. Recommended: use the OpenZeppelin implementations directly for your ERC-20 code.

OpenZeppelin ERC20 implementation:

IERC20

Interface of the ERC20 standard as defined in the EIP. Does not include the optional functions; to access them see ERC20Detailed.

FUNCTIONS

`totalSupply()` → `uint256`

`balanceOf(account)` → `uint256`

`transfer(recipient, amount)` → `bool`

allowance(owner, spender) → uint256

approve(spender, amount) → bool

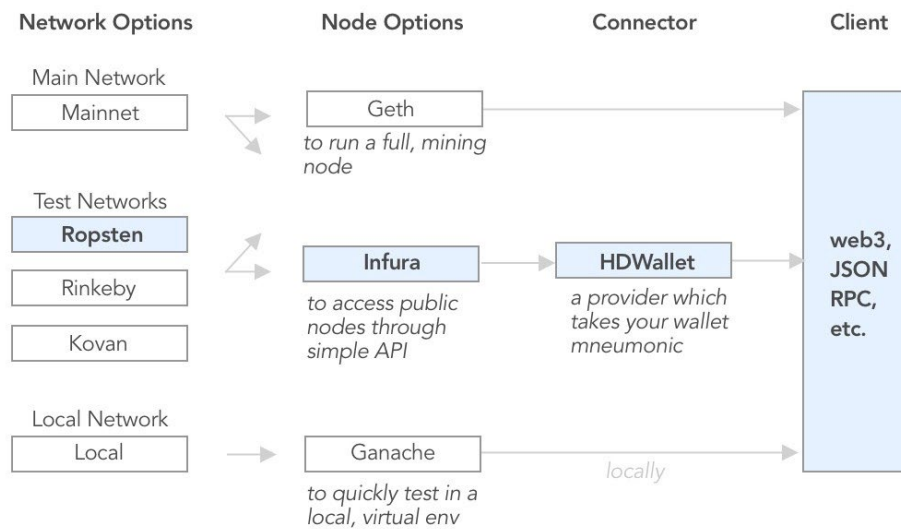
transferFrom(sender, recipient, amount) → bool

- These methods are shown in the previous screenshots

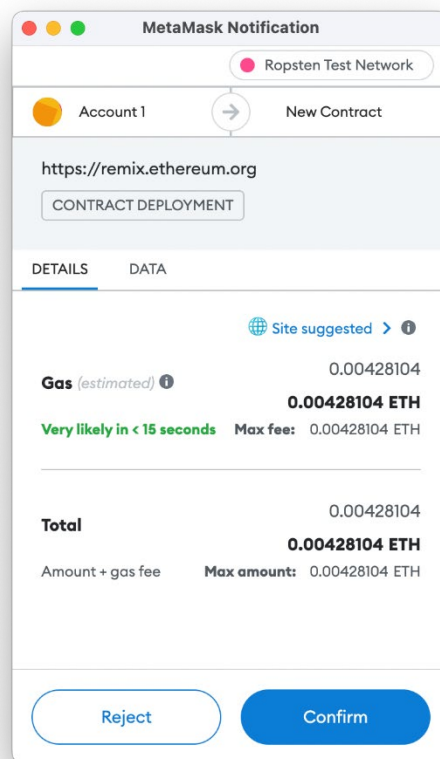
Marketplace and ERC-20 Integration

1. Deployed on Ropsten

Ethereum Smart Contract Deployment Options



2. Demonstrate the interaction of the marketplace with the ERC-20 token integration



MYTOKEN AT 0X661...D73A9 (BLOCK)

approve

spender: "0xFdf1419521E51E0662d76092"

amount: "1000"



transact

approveCustom address spender, uint256 am

decreaseAllo... address spender, uint256 sub

increaseAllow... address spender, uint256 add

transfer address recipient, uint256 am

transferFrom address sender, address recip

allowance

owner: 0x8D6e5378D050872803D34a6

spender: 0xFdf1419521E51E0662d76092



call

0: uint256: 1000

Transaction Details

Overview Internal Txns Logs (2) State

[This is a Ropsten Testnet transaction only]

Transaction Hash:	0x809104835b3ad73bc5dcda9dab0e365a758bd2e2d52d3a9857c6688af90587e7
Status:	Success
Block:	12250184 5 Block Confirmations
Timestamp:	2 mins ago (May-09-2022 01:38:05 AM +UTC)
From:	0x082fb06d10a0047270002e514d652d3c3b7a5b22
Interacted With (To):	Contract 0xdf1419521e51e0662d76092be1b8660bf63149c
Tokens Transferred:	From 0x082fb06d10a00... To 0xee663dd5c11ed... For 0 EVBToken (JKS)
Value:	0 Ether (\$0.00)
Transaction Fee:	0.00011490000128688 Ether (\$0.00)
Gas Price:	0.000000002500000028 Ether (2.500000028 Gwei)

3. All methods should now transact with the token instead of ETH

[Edit](#)
Ropsten Test Network

Account 1
Account 2

0xfb9...C8Ff : TRANSFER


50000 JKS

DETAILS DATA HEX

Market

Gas (estimated) 0.00011885
Likely in < 30 seconds Max fee: 0.00011885 ETH

Total 50000 JKS + 0.000119 ETH
Amount + gas fee Max amount: 50000 JKS + 0.000118848 ETH

Reject Confirm

4. Marketplace Dapp should connect and show the current ERC-20 token balance on the homepage

*The balance can be seen in the previous screenshots

5. The ERC-20 Token deployer should be able to transfer tokens to other users directly within the Dapp (from the homepage) as “airdrop” functionality.

*The airdrop functionality can be seen in the previous screenshots

6. You may also need other functionality to approve the Marketplace.sol contract to interact with the ERC-20.sol contract.

We have implemented a new smart contract which deals with the approve functionality provided by OpenZeppelin



```
pragma solidity ^0.8.9;

// SPDX-License-Identifier: MIT
// OpenZeppelin Contracts v4.3.2 (utils/Context.sol)

import "./tokenOpenZeppelin.sol";

contract MyToken is ERC20 {
    constructor() ERC20("EVBTOKEN", "JKS"){
        _mint(0x082fB06D10a0047270002e514D652d3c3b7a5B22, 1000000000 * (10 ** uint256(decimals())));
    }

    function approveCustom(address spender, uint256 amount) public virtual returns (bool) {
        _approve(_msgSender(), spender, amount);
        return true;
    }
}
```

Fig. Usage of Approve function

The approve function sets amount as the allowance of spender over the caller's tokens. It returns a boolean value indicating whether the operation succeeded. Let's assume sender wants to transfer tokens to a receiver, then the sender implicitly gives the permission to smart contract to retrieve the required amount from the sender's address.

1. Detailed explanation of ERC-20 functionality and methods and how the ERC-20 contract connects to the other contract and uses the functionality.

```
pragma solidity ^0.8.9;

// SPDX-License-Identifier: MIT
// OpenZeppelin Contracts v4.3.2 (utils/Context.sol)

import "./tokenOpenZeppelin.sol";

contract MyToken is ERC20 {
    constructor() ERC20("EVToken", "JKS"){
        _mint(0x082fB06D10a0047270002e514D652d3c3b7a5B22, 1000000000 * (10 ** uint256(decimals())));
    }

    function approveCustom(address spender, uint256 amount) public virtual returns (bool) {
        _approve(_msgSender(), spender, amount);
        return true;
    }
}
```

2. Detailed explanation of the overall deployment process for your contracts and Dapp connection.

The smart contract can be deployed as follows:

- Step 1: Deploy the token solidity file which has the ERC20 OpenZeppelin implementation
- Step 2: Deploy the main Charging.sol file
- Step 3: After that we approve the spender(smart contract) to spend on buyers behalf. This is usage of approve function.
- Step 4: We check if approve works by checking allowance function.
- Step 5: Once verified, we can go back to the main charging.sol file and check the buyCharging functionality to check whether the Buyer(EV Owner) can buy charge from the Seller(Charger Host).

- Step 6: We check the airdrop functionality using the functions specified in the solidity files.

3. **Original architectural diagram (can be of any design to illustrate the system and how it interacts with the other components - does not need to be UML) describing the ERC-20, Marketplace, Client, etc. interactions.**

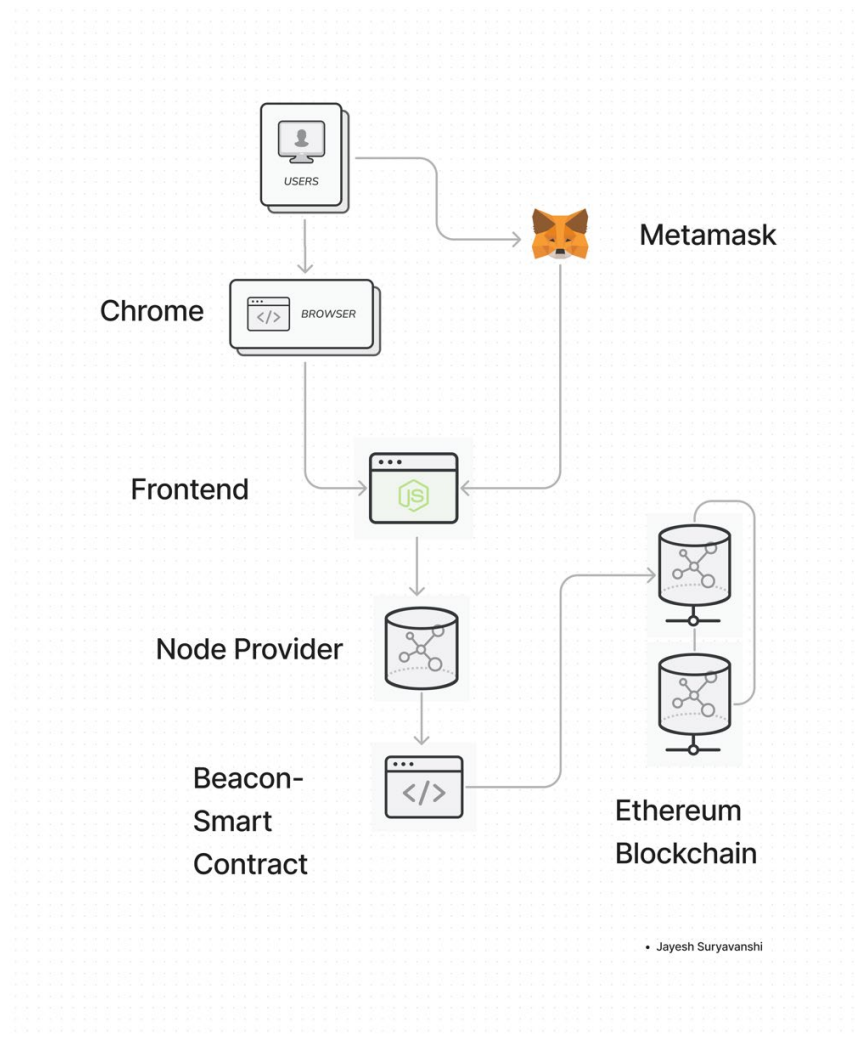
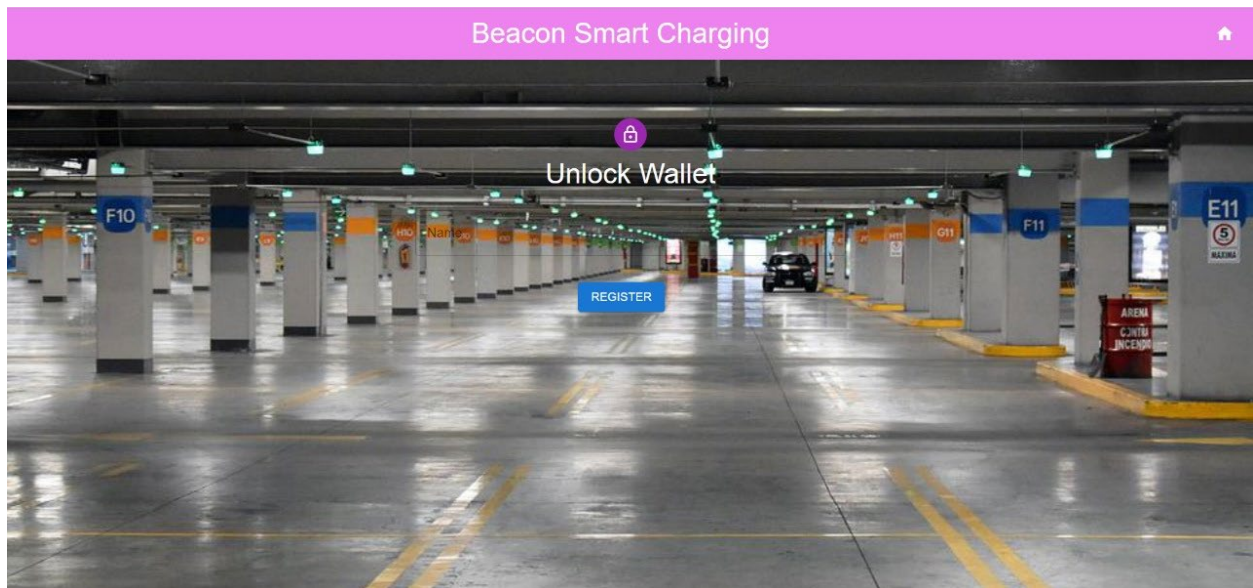


Fig. Architecture Diagram

Frontend Application Screenshots:



More screenshots to be added..